A **Tree** is a **non-linear data structure** used to represent hierarchical relationships.

Key Properties:

- Consists of nodes and edges
- One node is the root
- Every node can have children

```
   A
  / \
 B   C
/     \
D      E
```

Terminology:

- Root : Top node
- Parent / Child : Relationship between nodes
- Leaf Node : Node with no children
- Subtree : Tree inside a tree
- Depth : Distance from root
- Height : Longest path to leaf

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []

# Example creation
root = Node('A')
child1 = Node('B')
child2 = Node('C')

root.children.append(child1)
root.children.append(child2)
```

Uses of Trees

1. File Systems
- OS stores directories as trees

2. Database Indexing
- Trees like B-Trees, B+ Trees are used for fast searching

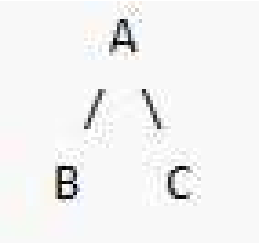
3. Searching Algorithms
- Binary Search Trees allow O(log n) search (average)

4. AI & Decision Making
- Decision Trees in Machine Learning

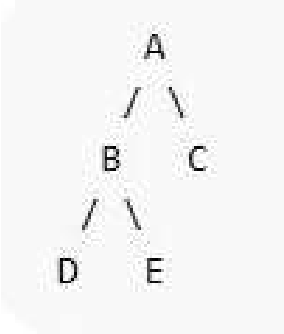
Shape of Binary Trees

A Binary Tree is a tree where each node has at most 2 children.

1. Full Binary Tree
Every node has 0 or 2 children



2. Complete Binary Tree
All levels filled except last
Last level filled left to right



3. Perfect Binary Tree
All internal nodes have 2 children
All leaves at same level

```
      A
    /   \
   B     C
  / \   / \
 D   E F   G
```

4. Skewed Binary Tree

Left Skewed:

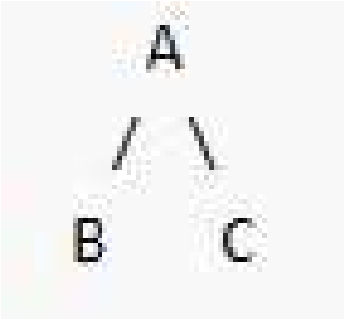
```
A
|
B
|
C
```

Right Skewed:

```
A
 \
  B
   \
    C
```

Python Binary Tree Node

```
class BinaryNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```



Three Common Applications of Binary Trees

1. Binary Search Tree (BST)

Left child < Root < Right child

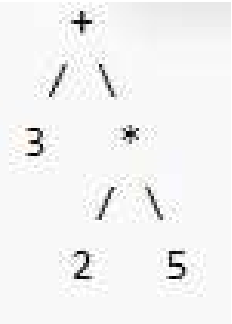
```
class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, key):
        if root is None:
            return BinaryNode(key)
        if key < root.data:
            root.left = self.insert(root.left, key)
        else:
            root.right = self.insert(root.right, key)
        return root
```

2. Expression Tree

Used to evaluate expressions

Example: (3 + (2 * 5))




```
def evaluate(root):
    if root.left is None and root.right is None:
        return int(root.data)

    left_val = evaluate(root.left)
    right_val = evaluate(root.right)

    if root.data == '+':
        return left_val + right_val
    elif root.data == '*':
        return left_val * right_val
```

3. Heap (Priority Queue)

Complete Binary Tree
Used in priority scheduling

Types:
Min Heap : smallest element at root
Max Heap : largest element at root

Code:

```
import heapq
heap = []
heapq.heappush(heap, 10)
heapq.heappush(heap, 5)
heapq.heappush(heap, 20)
print(heapq.heappop(heap))  # Output: 5
```

Uses:
Scheduling algorithms
Dijkstra’s algorithm
Priority queues

3. Heap (Priority Queue)

Complete Binary Tree
Used in priority scheduling

Types:
Min Heap : smallest element at root
Max Heap : largest element at root

Code:

```
import heapq
heap = []
heapq.heappush(heap, 10)
heapq.heappush(heap, 5)
heapq.heappush(heap, 20)
print(heapq.heappop(heap))  # Output: 5
```

Uses:
Scheduling algorithms
Dijkstra’s algorithm
Priority queues

Q1:

A tree with n nodes has how many edges?

A) n
B) n − 1
C) n + 1
D) 2n

Q2:

Which node has no children?

A) Root
B) Internal node
C) Leaf node
D) Parent node

Q5:

A binary tree in which all levels are filled except possibly the last, and nodes are filled from left is called:

A) Full Binary Tree
B) Perfect Binary Tree
C) Complete Binary Tree
D) Skewed Tree

The Three Main Traversals

1. Inorder Traversal (Left → Root → Right)

Algorithm:

1. Visit the left subtree.
2. Visit the root.
3. Visit the right subtree.

Output: 20 → 30 → 40 → 50 → 60 → 70 → 80

For Binary Search Trees, Inorder traversal gives the nodes in sorted ascending order.

2. Preorder Traversal (Root → Left → Right)

Algorithm:

1. Visit the root.
2. Visit the left subtree.
3. Visit the right subtree.

Output: 50 → 30 → 20 → 40 → 70 → 60 → 80

Preorder visits the root first, so it captures the tree structure.

3. Postorder Traversal (Left → Right → Root)

Algorithm:

1. Visit the left subtree.
2. Visit the right subtree.
3. Visit the root.

Output: 20 → 40 → 30 → 60 → 80 → 70 → 50

The root is visited after its children.

Deletion in BST

Deleting a node must still preserve the BST property. We handle three cases:

Case 1: Node to be deleted is a leaf (no children)

→ Just remove it.

Case 2: Node has one child

→ Replace the node with its child.

Case 3: Node has two children

→ Find Inorder Successor (smallest in right subtree), replace node's key with successor's key, and delete the successor.

Tree Transversal

Traversal = visiting all nodes in a specific order.

Type	Order	Output Meaning
Inorder	Left → Root → Right	Sorted Order
Preorder	Root → Left → Right	Tree Construction
Postorder	Left → Right → Root	Node Deletion

Time Complexities:

Operation	Average	Worst Case (skewed tree)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)
Search	O(log n)	O(n)

What is Parsing?

Parsing = analyzing a string according to grammar rules

Used in:

- Compilers
- Interpreters
- Calculators

Real-life Example: 3 + 5 * 2

Parsing decides:

- Order of evaluation
- Structure of expression

2. Types of Expressions

1. Infix Expression

Operator between operands

- A + B

2. Prefix Expression

- + A B

3. Postfix Expression

- A B +

3. Operator Precedence

Operator	Priority
^	High
*, /	Medium
+, -	Low

4. Infix → Postfix Conversion

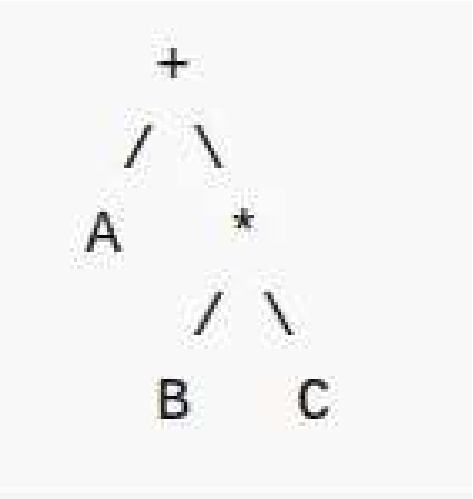
Algorithm (Stack-based)

- Scan expression left to right
- If operand → output
- If operator → push to stack
- If higher precedence → push
- If lower → pop until condition satisfied
- If (→ push
- If) → pop until (

Expression Trees

1. What is an Expression Tree?
A binary tree representation of expressions
Leaves : operands
Internal nodes : operators

Expression: A + B * C



Node Structure in Python

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

3. Construct Expression Tree from Postfix

Algorithm

1. Scan postfix expression
2. If operand → push node
3. If operator:
    - Pop two nodes
    - Create new node
    - Push back

Hash Table

Hashing is a technique used to map data (keys) to a fixed-size table (array) called a hash table, using a mathematical function known as a hash function.

It provides fast insertion, deletion, and search operations — typically in O(1) average time.

Hash Table ADT (Abstract Data Type)

A Hash Table is an implementation of an array, which stores data as (key, value) pairs.

Basic Operations

1. insert(key, value) - Stores a key-value pair.
2. delete(key) - Removes an entry with the specified key.
3. search(key) - Retrieves the value associated with the key.

Hash Table ADT (Abstract Data Type)

Key	Hash Value (h(key))	Index Value	Stored Value
12	h(12) = 2	2	"Apple"
44	h(44) = 4	4	"Mango"

Components of Hashing

1. Keys: The unique identifiers of data elements.
2. Hash Function: Maps keys to table indices.
3. Hash Table: The array or storage where values are placed.
4. Collision Resolution Mechanism: Handles cases when two keys map to the same index.
5. Load Factor (α): Measures how full the hash table is.

Hash Table

A Hash Table is a fixed-size array H[0 . . m -1]
where:

Each position (bucket) stores a record or linked list of records.

The index for a key k is computed as h(k).

Hash Table

Example:

Let $m = 10$ (table size)

Hash function: $h(k) = k \% 10$

The hash function $h(k)$ determines the index for each key in the table.

A good hash function distributes keys uniformly across all indices.

Hash Table

Keys: 12, 25, 35, 26

$h(12)=2 \rightarrow H[2]$

$h(25)=5 \rightarrow H[5]$

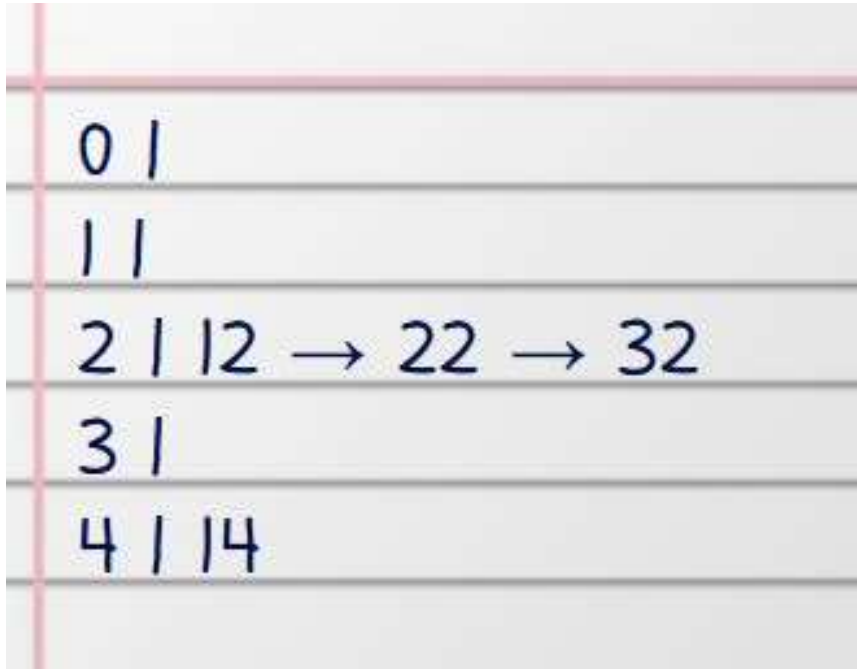
$h(35)=5 \rightarrow H[5]$ (collision)

$h(26)=6 \rightarrow H[6]$

A collision happens when two keys map to the same index in the hash table.

1. Separate Chaining (Closed Hashing)

Each index stores a **linked list** of all keys that map there.



If multiple keys go to the same index, we just **chain them together** in a linked list.

Slightly more memory (for pointers)

2. Open Addressing

Unlike separate chaining, **all elements are stored directly in the hash table** itself.
If a collision happens, the algorithm **finds another empty slot** in the table — this is calle **probing**.

No linked lists or extra pointers are used here.

1. Compute hash: `index = h(key)`.
2. If `table[index]` is empty → insert key there.
3. If `table[index]` is full → use a **probing method** to find the next available slot.

Linear Probing

$$h(k,i)=(h(k)+i)\%m$$

Start with `i = 0` → check `h(k)`.

If full, `i = 1` → check `h(k)+1`.

Repeat until an empty slot is found.

Linear Probing

Example

Hash table size = 7, $h(k) = k \% 7$
Keys to insert: 10, 20, 15

Insert 10 → $h(10) = 10 \% 7 = 3$ → slot 3 empty → insert.
Insert 20 → $h(20) = 20 \% 7 = 6$ → slot 6 empty → insert.
Insert 15 → $h(15) = 15 \% 7 = 1$ → slot 1 empty → insert.
Insert 24 → $h(24) = 24 \% 7 = 3$ → slot 3 occupied → check 4 → empty → insert at 4.

Quadratic Probing

$$h(k,i) = (h(k) + i^2)\%m$$

Instead of checking consecutive slots, jump **quadratically**: 1^2, 2^2, 3^2…

Reduces clustering compared to linear probing.

Quadratic Probing

Example:

Insert 10, 20, 24 into a table of size 7

Insert 10 → index 3

Insert 24 → collision at 3 → try $3 + 1^2 = 4$ → empty → insert at 4

Double Hashing

$$h(k,i) = (h_1(k) + i \cdot h_2(k))\%m$$

Use **two hash functions**: h1 for initial index, h2 for step size.

Reduces clustering **even more** than quadratic probing.

Double Hashing

Example:

$h1(k) = k \% 7$

$h2(k) = 1 + (k \% 5)$

Insert key 24:

First try: $h1(24) = 24 \% 7 = 3 \rightarrow$ collision

Next try:

$3 + 1h2(24) = 3 + 1\ (1 + 24 \% 5)$

$= 3 + 1 * 5 = 8 \rightarrow$ wrap around $\rightarrow$ index 1

Comparison of Collision Resolution Techniques

Method	Idea	Extra Space	Problem	Example/Use
Separate Chaining	Linked list at each index	Yes	Slower with long chains	Flexible memory use, good when memory is not tight
Linear Probing	Check next slot	No	Primary clustering	Simple, fast for low load
Quadratic Probing	Jump in squares	No	Secondary clustering	Medium load factor
Double Hashing	Use 2 hash functions	No	More complex to compute	Best performance, fewer collisions